

EC4010
DIGITAL DESIGN
ANALYSIS AND DESIGN OF
ASYNCHRONOUS SEQUENTIAL
CIRCUITS

BANDARA H.G.T.D.

2022/E/048

GROUP CG04

SEMESTER 04

20 JAN 2025

TITLE: ANALYSIS AND DESIGN OF ASYNCHRONOUS SEQUENTIAL CIRCUITS

OBJECTIVES:

- Understand and apply basic Verilog HDL coding.
- Understand difference between behavioral and structural modeling.
- Identifying hazards in a combination logic circuit.
- Model an asynchronous sequential circuits and identify race conditions and stability considerations.

TASK 01:

Simulation Using Block Diagram

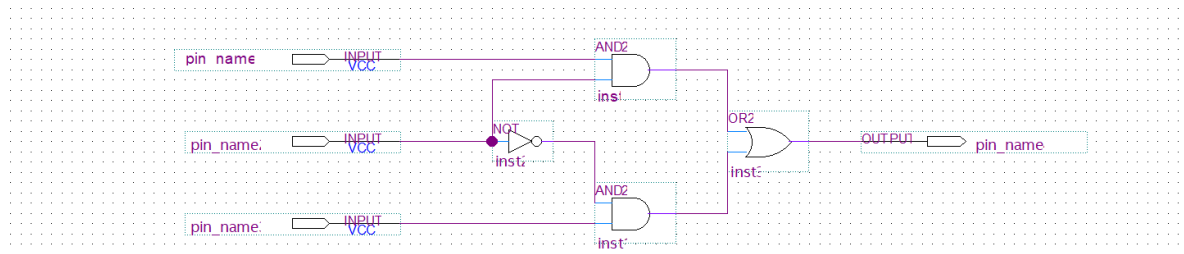


Figure 01: Combinational Circuit

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Jan 20 19:19:30 2025
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Lite Edition
Revision Name	lab05
Top-level Entity Name	lab05
Family	Cyclone 10 LP
Total logic elements	1 / 6,272 (< 1 %)
Total registers	0
Total pins	4 / 89 (4 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)
Device	10CL006YE144C6G
Timing Models	Final

Figure 02: Compilation Report

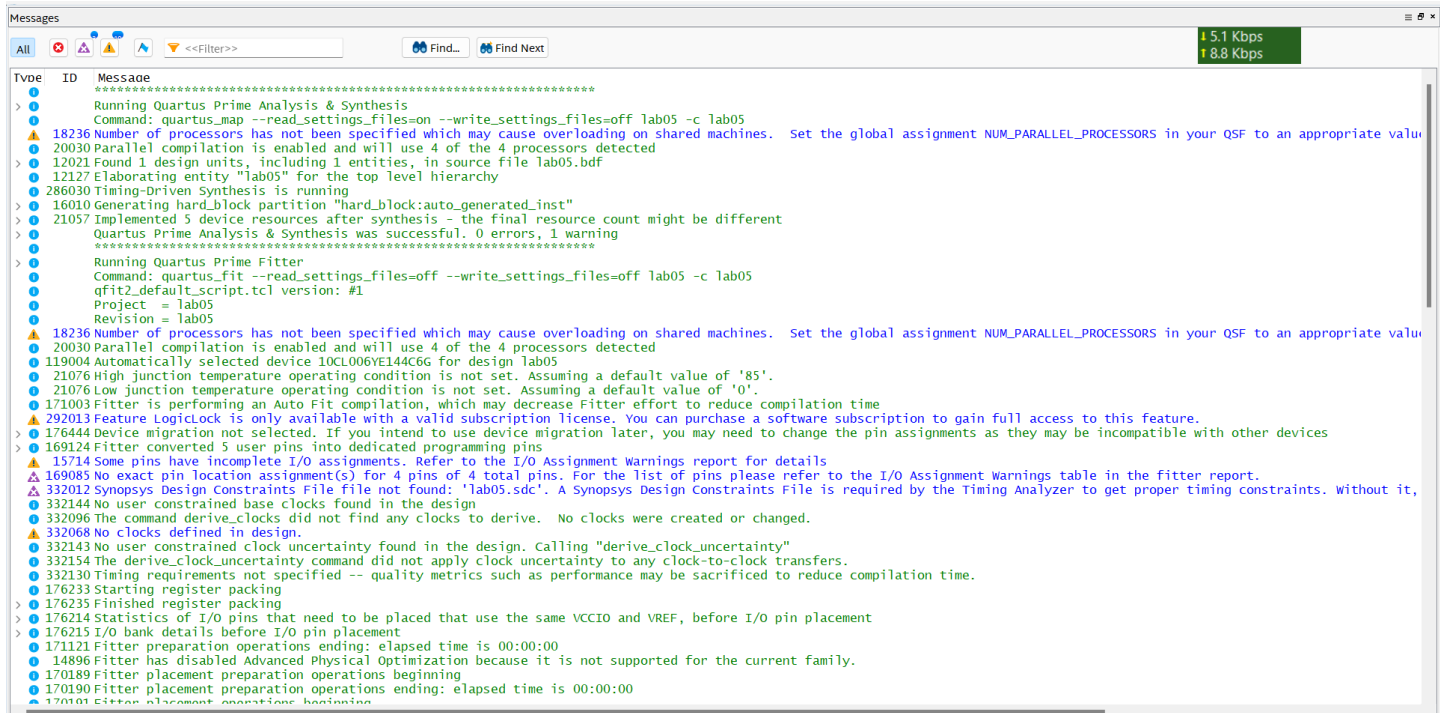


Figure 03

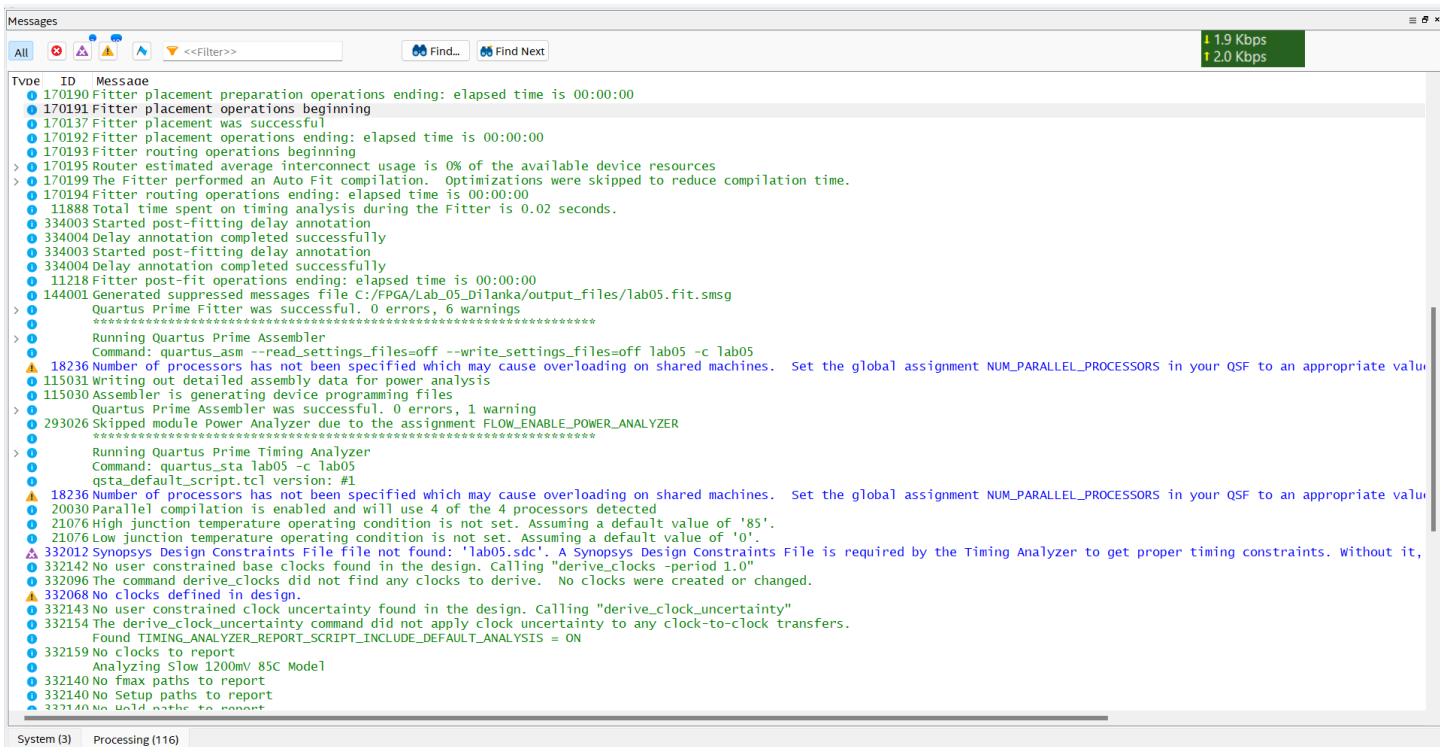


Figure 04

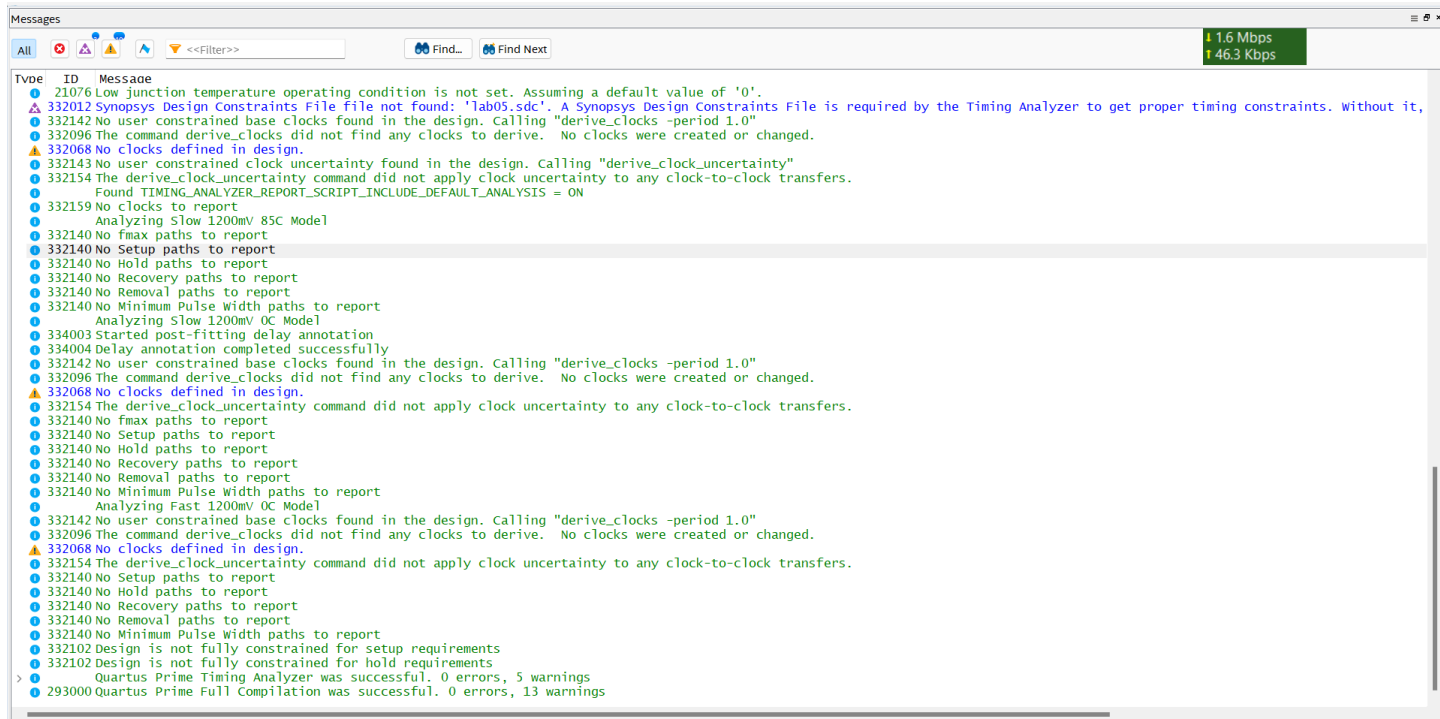


Figure 05

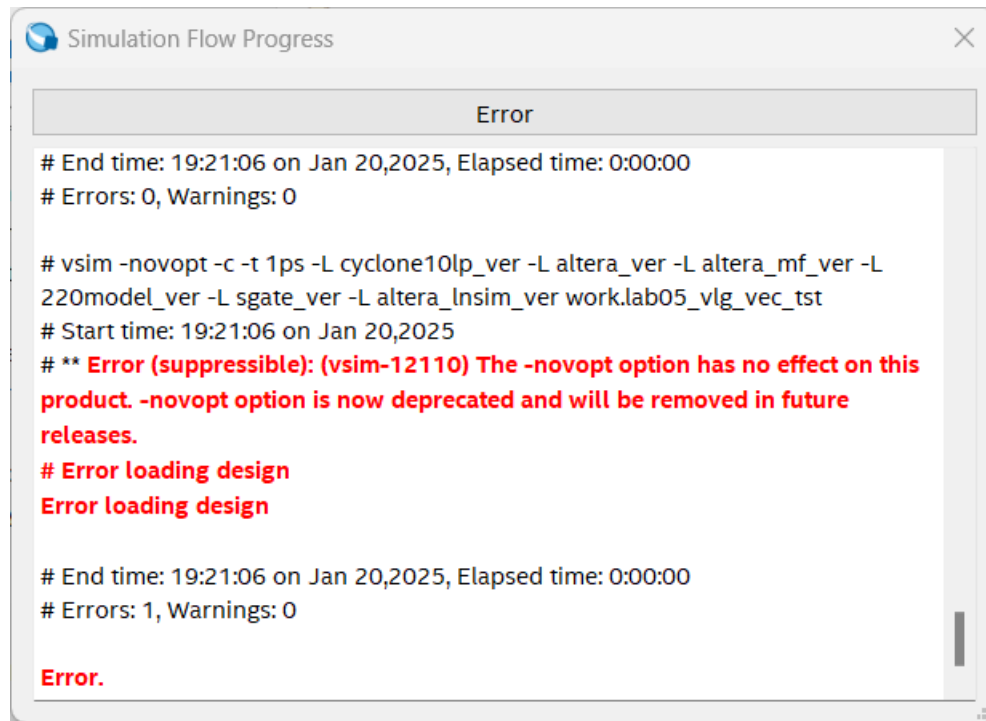


Figure 06: NOVoPT Error

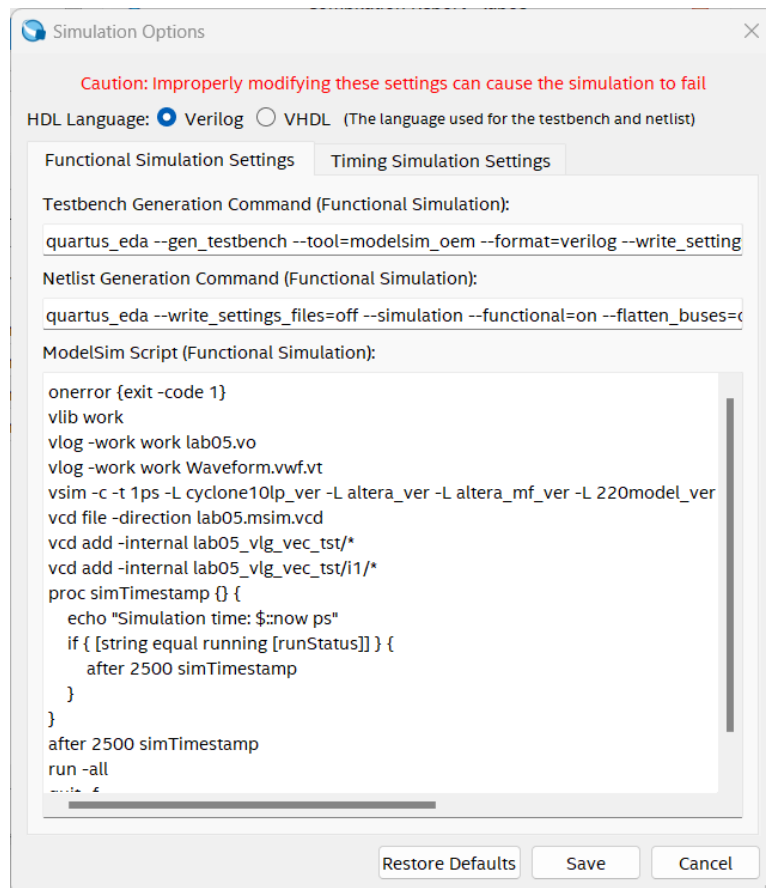


Figure 07: Solve NOVoPT Error

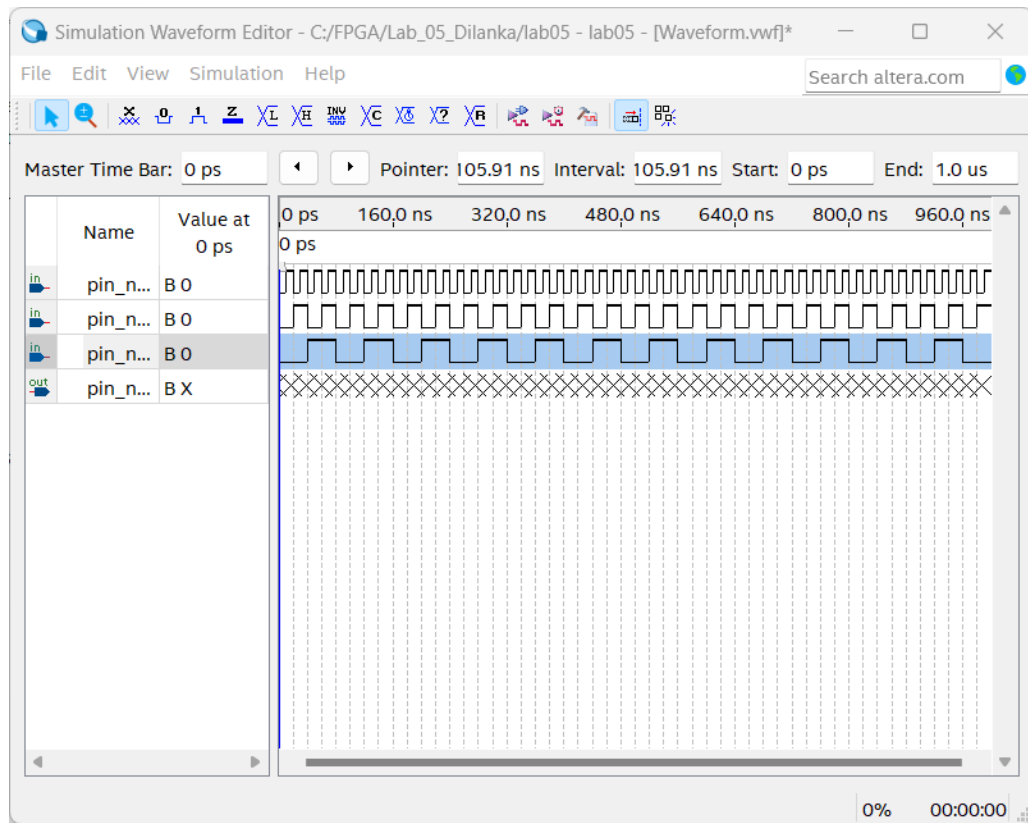


Figure 08: Before the Simulation

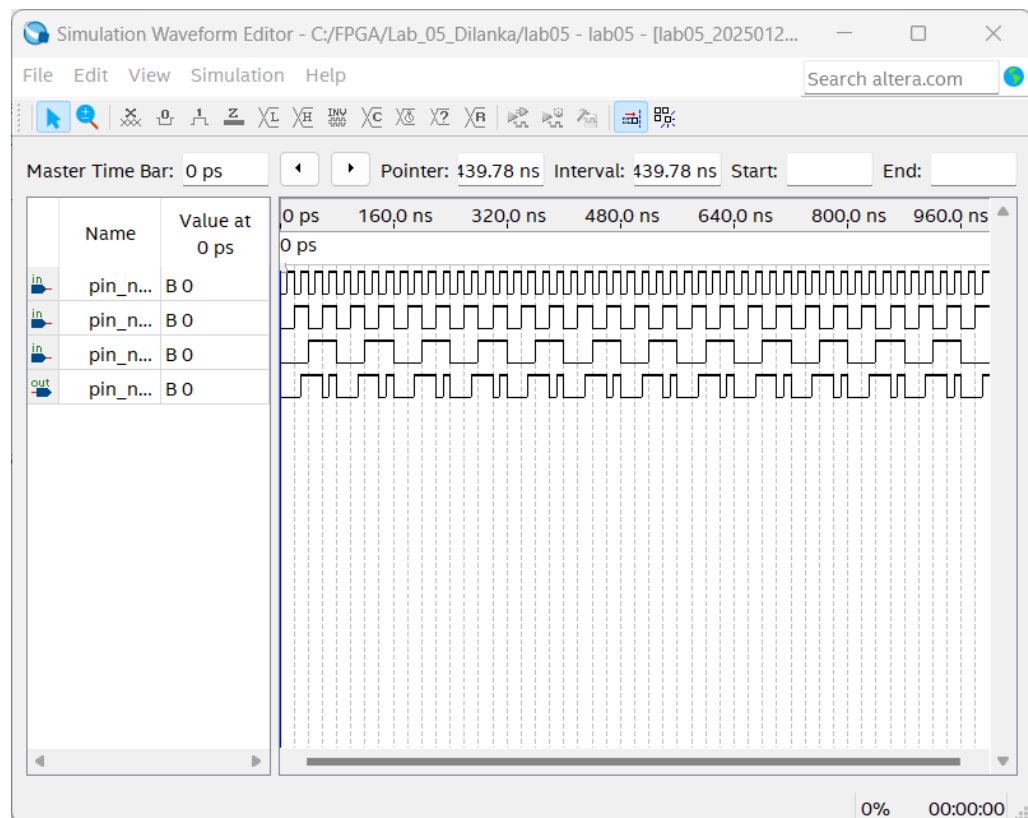


Figure 09: After the Simulation

TASK 02:

Simulation Using Verilog with Gate Delays

```
lab05.bdf
Compilation Report - la

1 module logic (
2
3     //inputs
4     input X,
5     input Y,
6     input Z,
7
8     //output
9     output Output
10
11 );
12
13 //wires
14 wire and1_out;
15 wire and2_out;
16 wire not_out;
17
18 //gates
19 not u1 (not_out, Y);
20 and u2 (and1_out, X, Y);
21 and u3 (and2_out, not_out, Z);
22 or u4 (Output, and1_out, and2_out);
23
24 endmodule
```

Figure 10: Code

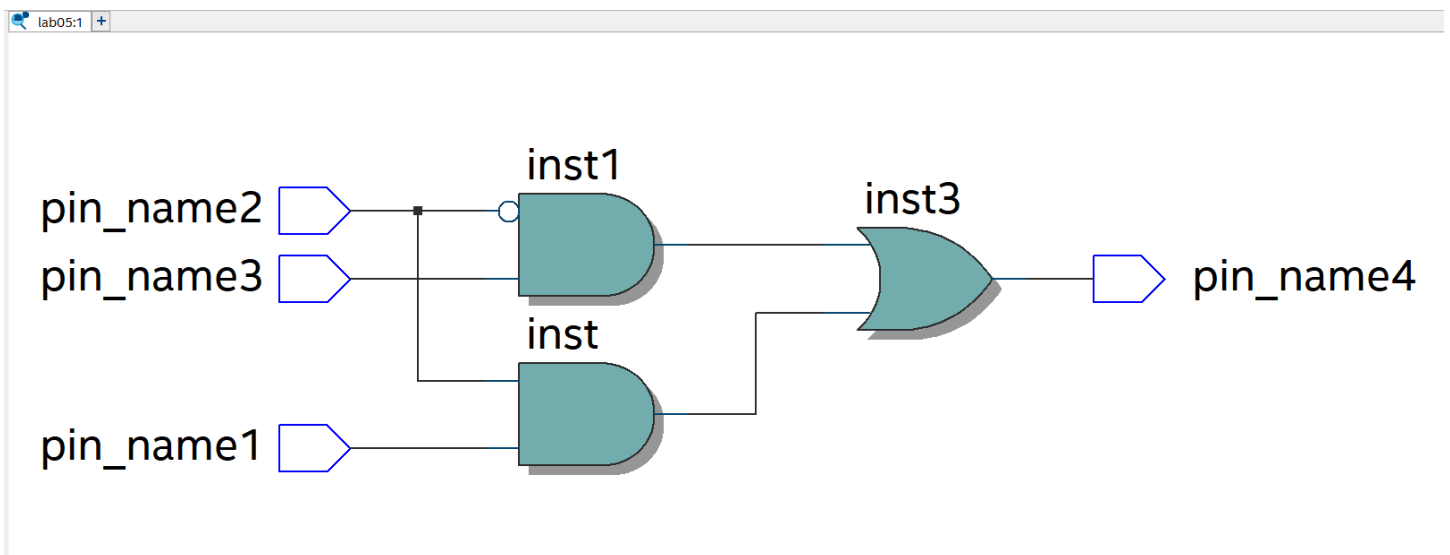


Figure 11: Circuit Diagram

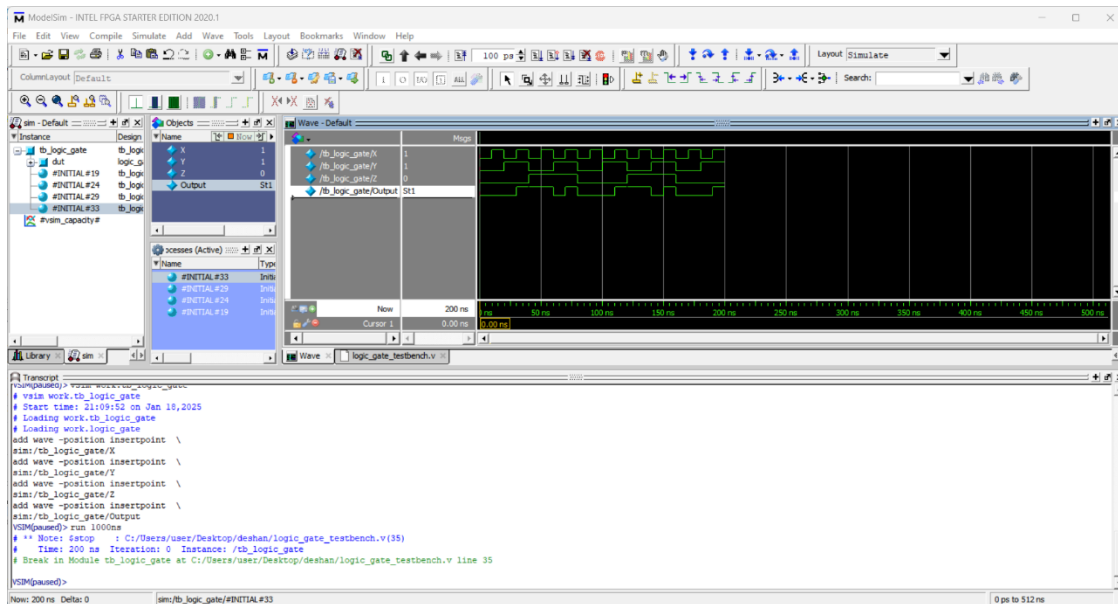


Figure 12: Output Waveform

Wave Comparison:

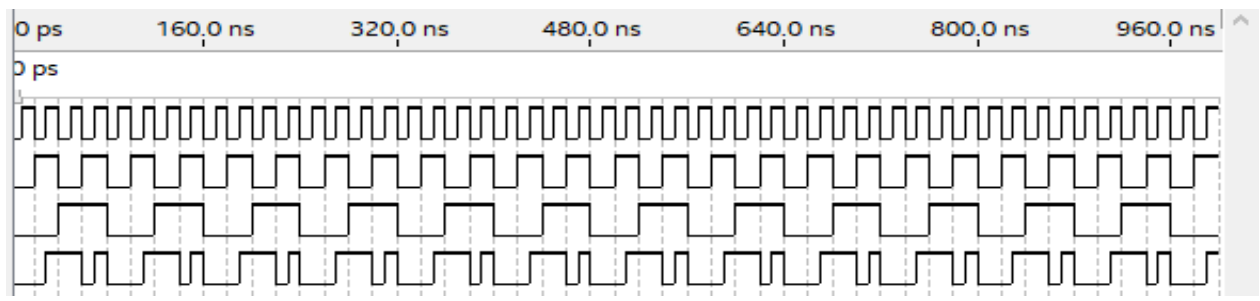


Figure 13: Using Block /Schematic

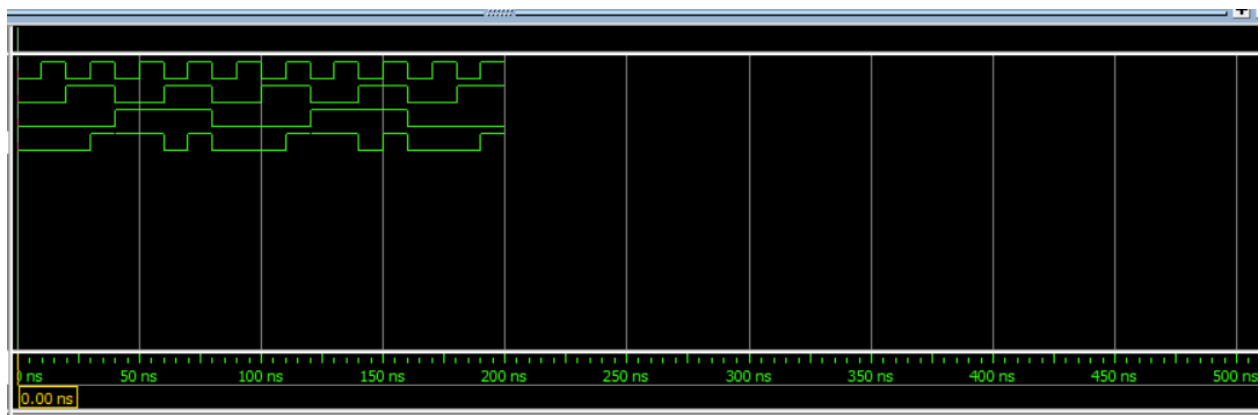


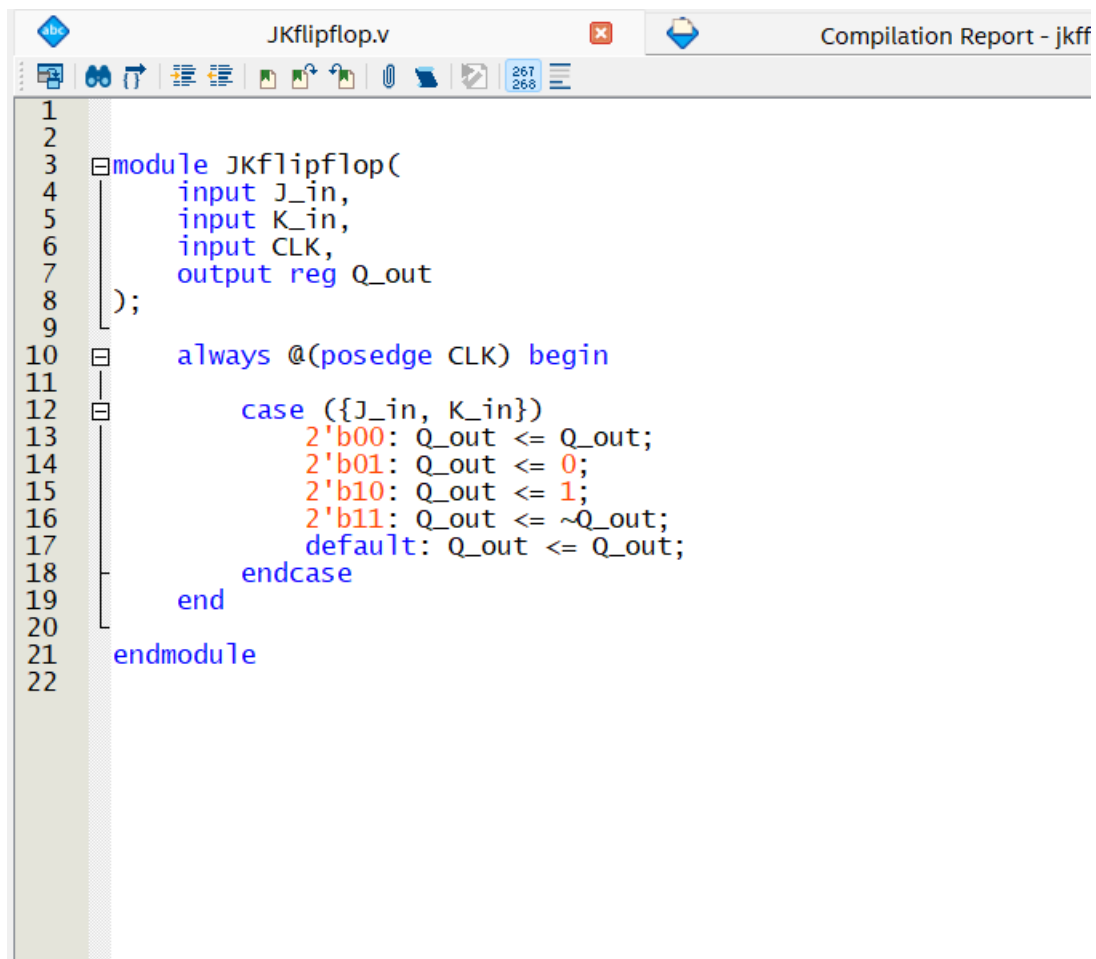
Figure 13: Using Verilog HDL

Conclusion:

Using Quartus software, we implemented a logic gate through the Block Diagram Schematic method and also coded its equivalent design using Verilog HDL. After simulating both implementations, we observed identical output waveforms, confirming the correctness of our design. This demonstrates that both schematic-based and HDL-based approaches can achieve the same functional results in digital design. It highlights the flexibility in choosing a design method, as both techniques, when correctly implemented, produce reliable and accurate circuit behavior. This verification reinforces the idea that digital circuits can be designed using different methodologies while maintaining the same functionality.

Q2

JK FlipFlop



The image shows a screenshot of the Quartus II software interface. The main window displays the Verilog HDL code for a JK FlipFlop, named 'JKflipflop.v'. The code is as follows:

```
1
2
3 module JKflipflop(
4     input J_in,
5     input K_in,
6     input CLK,
7     output reg Q_out
8 );
9
10 always @(posedge CLK) begin
11
12     case ({J_in, K_in})
13         2'b00: Q_out <= Q_out;
14         2'b01: Q_out <= 0;
15         2'b10: Q_out <= 1;
16         2'b11: Q_out <= ~Q_out;
17         default: Q_out <= Q_out;
18     endcase
19 end
20
21 endmodule
22
```

The code defines a module 'JKflipflop' with inputs 'J_in', 'K_in', and 'CLK', and an output 'Q_out' of type 'reg'. It uses an 'always' block triggered by the positive edge of 'CLK' to implement the JK flip-flop logic using a 'case' statement. The 'case' statement handles four input combinations: '00' (no change), '01' (set to 0), '10' (set to 1), and '11' (toggle). A 'default' case is also provided for safety. The code is well-formatted with line numbers on the left and syntax highlighting.

Figure 14: Verilog HDL Code

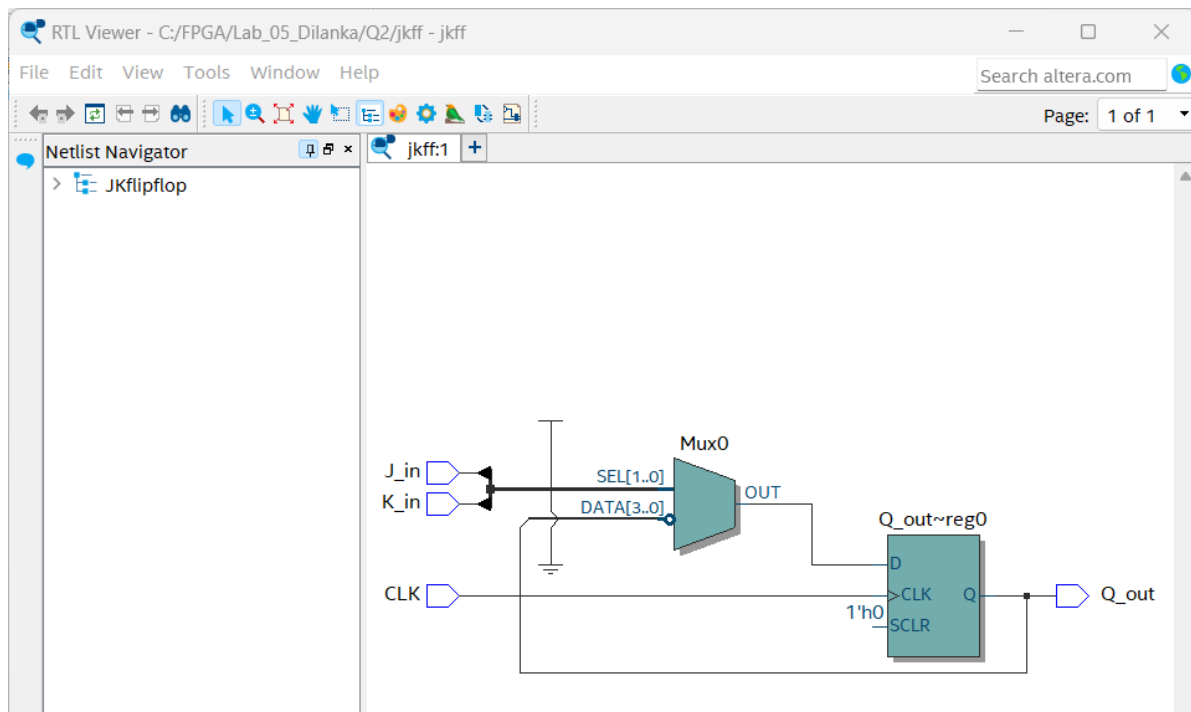


Figure 15: JK ff Diagram

JKflip.v

Table of Contents

Flow Summary

Flow Settings

Flow Non-Default Global Settings

Flow Elapsed Time

Flow OS Summary

Flow Log

Analysis & Synthesis

Fitter

Assembler

Timing Analyzer

EDA Netlist Writer

Flow Messages

Flow Suppressed Messages

Compilation Report - jkff

Flow Summary

<<Filter>>

Flow Status

Successful - Sat Jan 18 22:51:31 2025

Quartus Prime Version

20.1.0 Build 711 06/05/2020 SJ Lite Edition

Revision Name

jkff

Top-level Entity Name

JKflip

Family

Cyclone 10 LP

Total logic elements

1 / 6,272 (< 1 %)

Total registers

1

Total pins

4 / 89 (4 %)

Total virtual pins

0

Total memory bits

0 / 276,480 (0 %)

Embedded Multiplier 9-bit elements

0 / 30 (0 %)

Total PLLs

0 / 2 (0 %)

Device

10CL006YE144C6G

Timing Models

Final

Figure 16: Compilation_Report

```

1 timescale 1ns / 1ps
2
3 module JKff_tb;
4     reg J_in, K_in, CLK;
5     wire Q_out;
6
7     reg [1:0] testvectors [7:0];
8     integer i;
9
10
11
12     initial begin
13         testvectors[0] = 2'b10; // Set: Q = 1
14         testvectors[1] = 2'b00; // Hold: No change in Q
15         testvectors[2] = 2'b01; // Reset: Q = 0
16         testvectors[3] = 2'b00; // Hold: No change in Q
17         testvectors[4] = 2'b10; // Set: Q = 1
18         testvectors[5] = 2'b11; // Toggle: Q flips
19         testvectors[6] = 2'b01; // Reset: Q = 0
20         testvectors[7] = 2'b11; // Toggle: Q flips
21         i = 0; // Initialize index
22     end
23
24     JKflip JKflip_dut(
25         .J_in(J_in),
26         .K_in(K_in),
27         .CLK(CLK),
28         .Q_out(Q_out)
29     );
30
31
32     always begin
33         CLK = 0;
34         #5; // Low for 5 ns
35         CLK = 1;
36         #5; // High for 5 ns
37     end
38

```

Figure 17: Test Bench Code

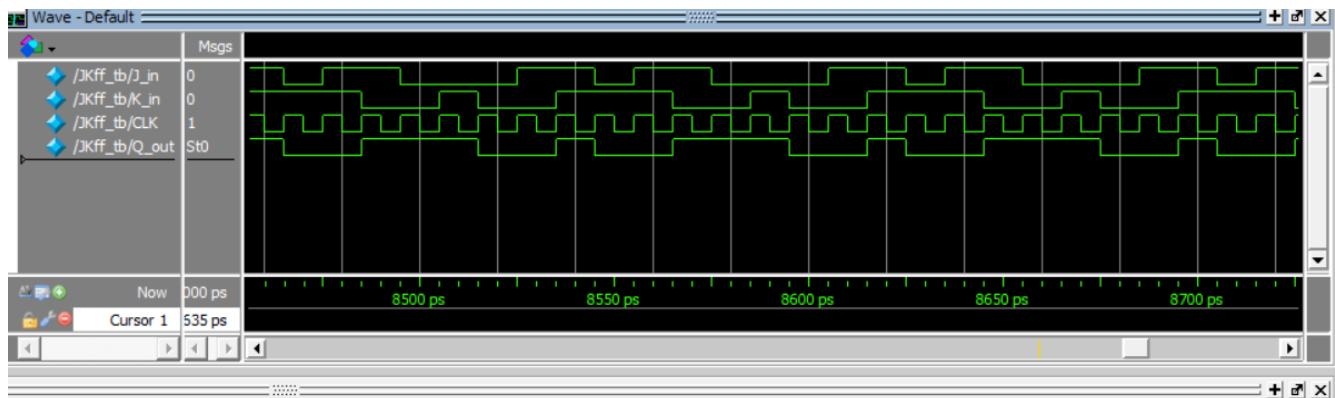


Figure 18: Output Waveform

T FlipFlop

```
module Tflip(  
    input T_in,  
    input CLK,  
    input rst_in,  
    output reg Q_out  
);  
  
    always @(posedge CLK or posedge rst_in) begin  
        if (rst_in)  
            Q_out <= 0; // Reset  
        else if (T_in)  
            Q_out <= ~Q_out; // Toggle  
        else  
            Q_out <= Q_out; // Hold  
    end  
endmodule
```

Figure 19: Verilog HDL Code

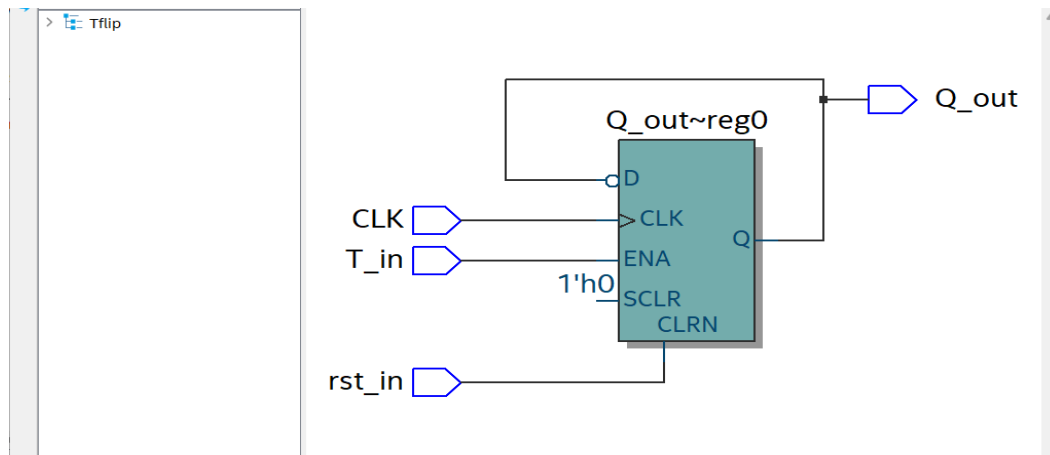


Figure 20: T ff Diagram

Table of Contents

Flow Summary

Flow Settings

Flow Non-Default Global Settings

Flow Elapsed Time

Flow OS Summary

Flow Log

Analysis & Synthesis

Fitter

Flow Messages

Flow Suppressed Messages

Assembler

Timing Analyzer

Flow Summary

<<Filter>>

Flow Status

Successful - Sat Jan 18 23:03:01 2025

Quartus Prime Version

20.1.0 Build 711 06/05/2020 SJ Lite Edition

Revision Name

tff

Top-level Entity Name

Tflip

Family

Cyclone 10 LP

Total logic elements

1 / 6,272 (< 1 %)

Total registers

1

Total pins

4 / 89 (4 %)

Total virtual pins

0

Total memory bits

0 / 276,480 (0 %)

Embedded Multiplier 9-bit elements

0 / 30 (0 %)

Total PLLs

0 / 2 (0 %)

Device

10CL006YE144C6G

Timing Models

Final

Figure 21: Compilation_Report

```

1  module tfflptb();
2
3      reg T_in, rst_in, CLK;
4      wire Q_out;
5
6      reg [2:0] testvectors [4:0];
7      integer i = 0;
8
9      initial begin
10         // Initialize test vectors for T_in and rst_in inputs
11         testvectors[0] <= 2'b10; // Reset
12         testvectors[1] <= 2'b00; // Hold
13         testvectors[2] <= 2'b01; // Toggle
14         testvectors[3] <= 2'b00; // Hold
15         testvectors[4] <= 2'b01; // Toggle
16     end
17
18     // Instantiate the T flip-flop
19     Tflip Tflip_dut(
20         .T_in(T_in),
21         .CLK(CLK),
22         .rst_in(rst_in),
23         .Q_out(Q_out)
24     );
25
26     // Clock generation
27     always begin
28         CLK = 1'b0;
29         #5;
30         CLK = 1'b1;
31         #5;
32     end
33
34     // Apply test vectors to the inputs
35     always @(posedge CLK) begin
36         if (i == 5) begin
37             i = 0; // Reset the index after 5 test vectors
38         end
39         {rst_in, T_in} = testvectors[i]; // Apply current test vector
40         i = i + 1;
41     end
42 endmodule
43
44

```

Figure 22: Test Bench Code

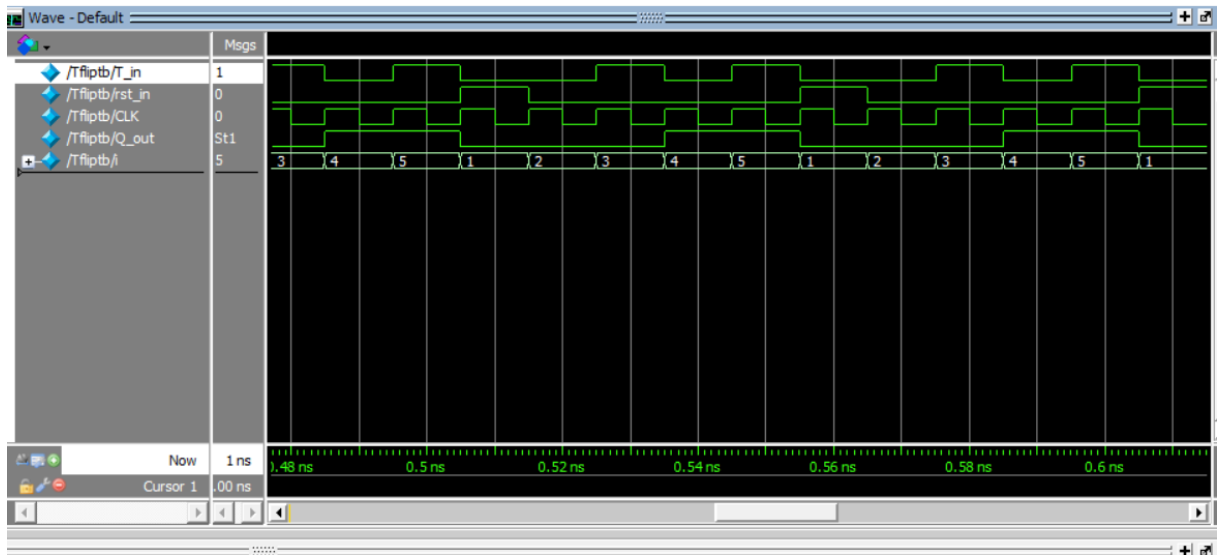


Figure 23: Output Waveform